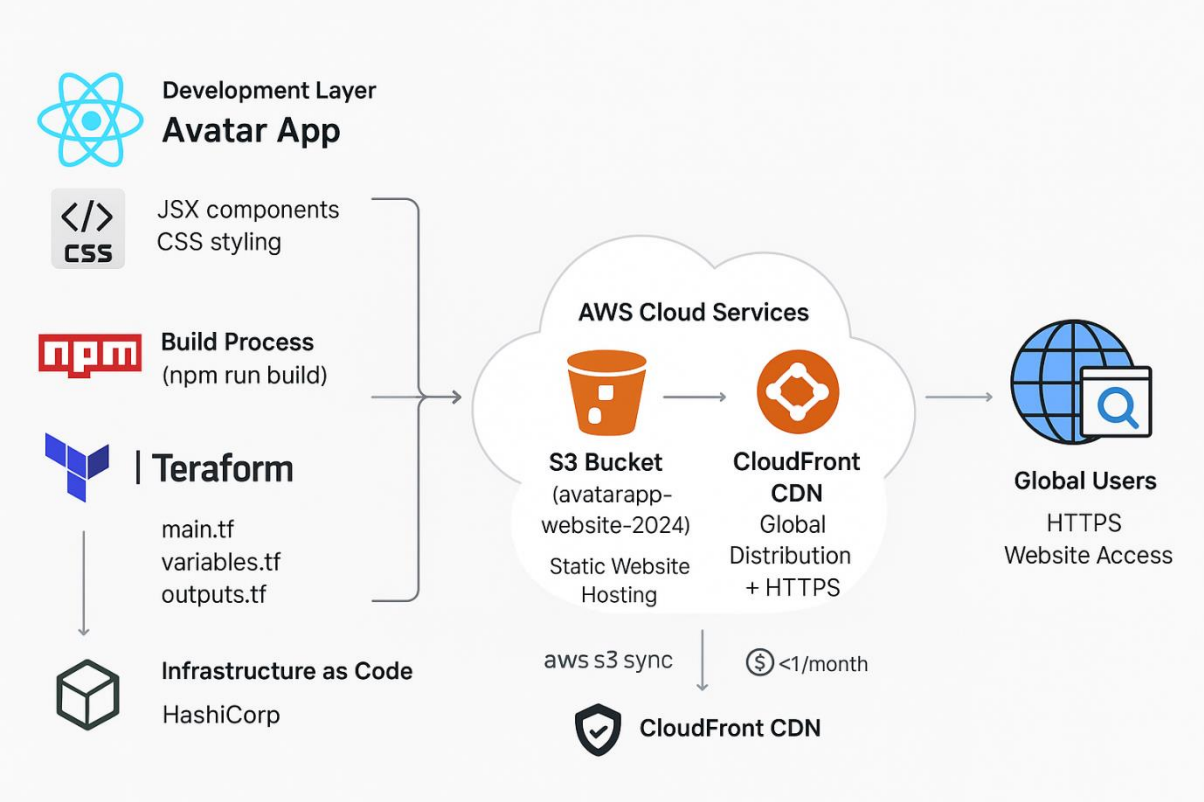
Avatar App – React Website Using IAC(AWS+Terraform)

1. Introduction

The **Avatar App** is a React-based web application that allows users to display and interact with customizable avatar components. This project demonstrates both frontend development using React and deployment of a full-fledged web application on **AWS** using **Terraform** for infrastructure automation.

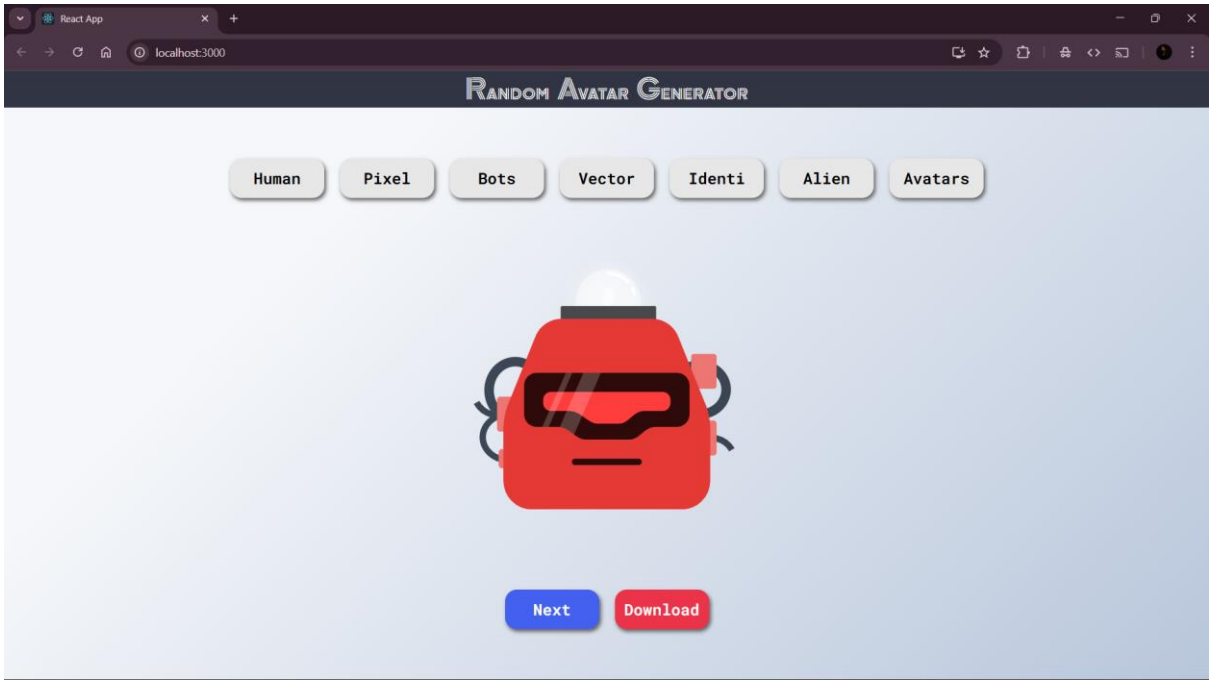
Architecture:



By combining React, AWS S3, and CloudFront with Terraform, this project highlights modern **DevOps practices**, including Infrastructure as Code (IaC), CI/CD readiness, and scalable deployment strategies.

Key Features:

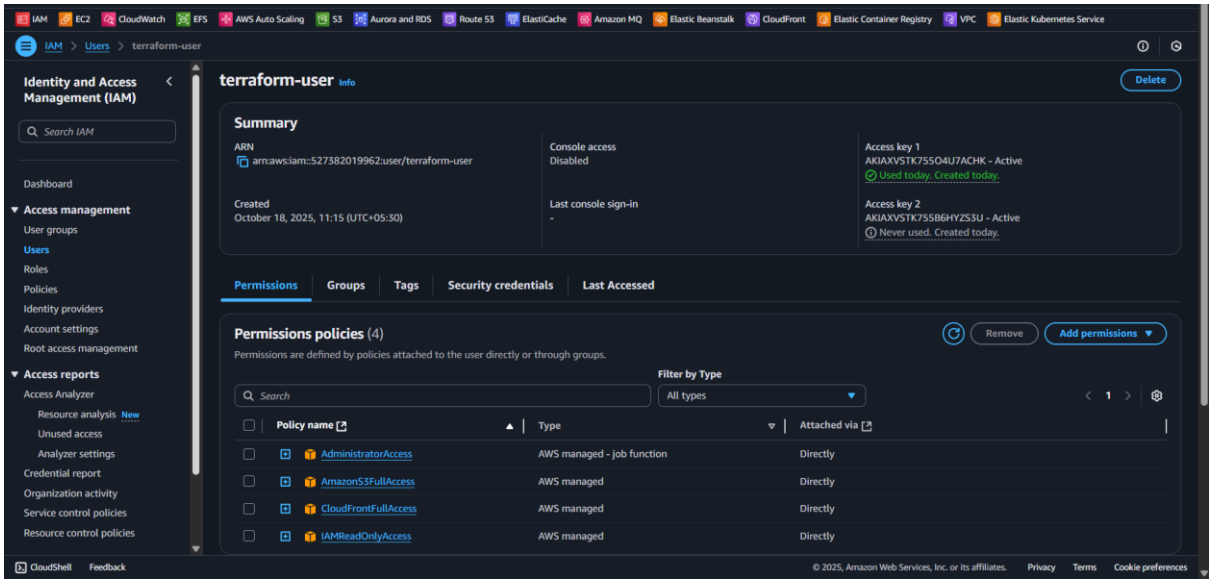
- Interactive Avatar components in React.
- Responsive UI with clean styling.
- Full deployment to AWS with HTTPS support.
- Infrastructure managed via Terraform for reproducibility.



2. Prerequisites

Before setting up the project, ensure the following tools and accounts are available:

1. **Local Development**
   - Node.js (v16+ recommended)
   - npm (comes with Node.js)
   - Git (optional, for cloning repo)
2. **AWS Setup**
   - AWS account with permissions to create S3 buckets, CloudFront distributions, and IAM roles.



   - AWS CLI installed and configured:

    - `aws configure`
    - Terraform installed (v1.x+ recommended)
    - sssssssssssssssssss
3. **Other**
    - Text editor/IDE (VSCode, WebStorm)
    - Internet connection for package downloads and deployment

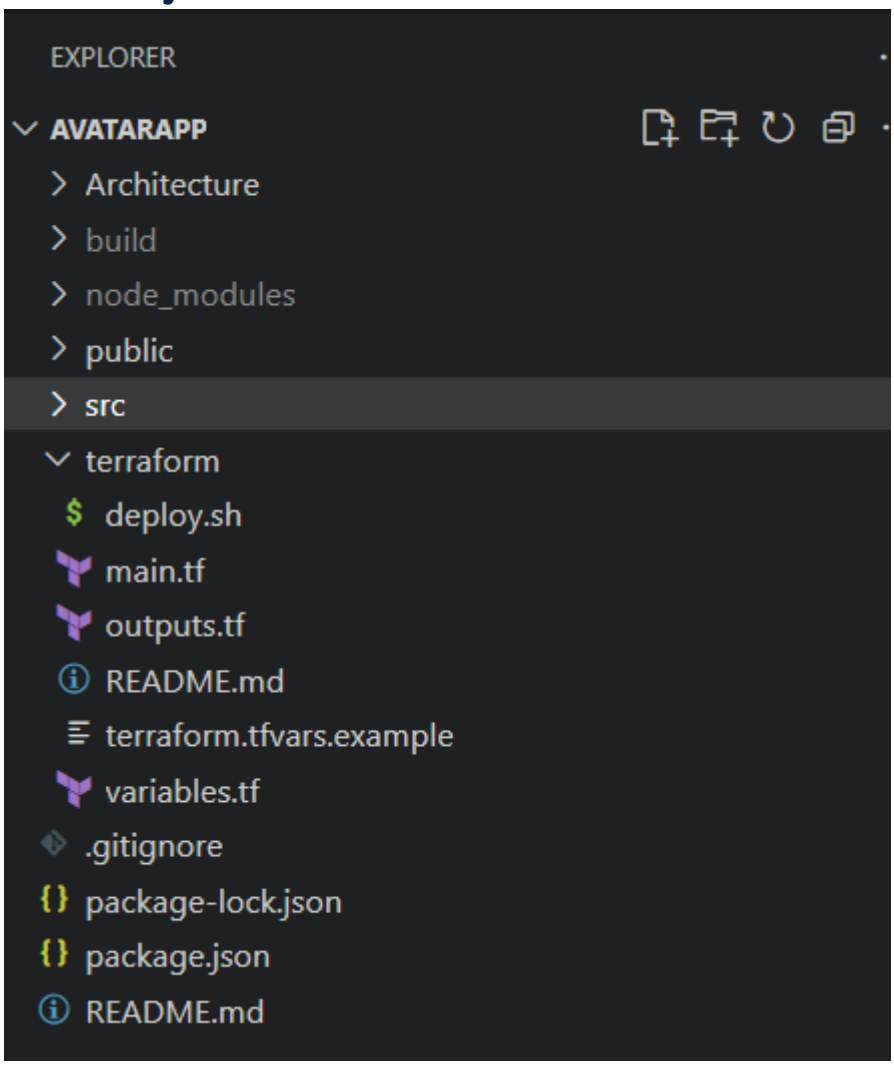
3. Why Terraform?

Terraform is used in this project to **automate infrastructure setup**. The benefits include:

- **Infrastructure as Code (IaC):** All AWS resources are defined declaratively.
- **Reproducibility:** Easily create or destroy environments without manual intervention.
- **Version Control:** Infrastructure changes can be tracked in Git along with the app.
- **Scalability:** Works seamlessly with AWS S3, CloudFront, and other services.

By using Terraform, we avoid manually creating S3 buckets, setting CloudFront distributions, or configuring permissions, making deployment consistent and error-free.

4. Project Structure



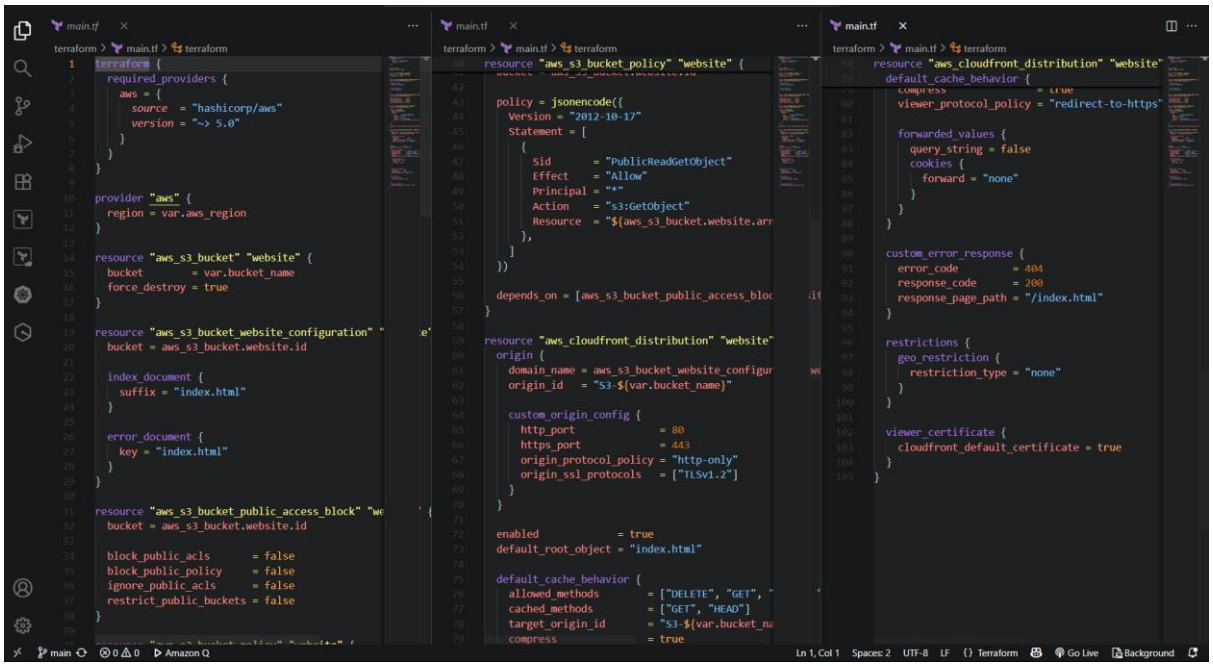
- `src/Components`: Contains React components, including Avatar.jsx.
- `src/Styles`: Contains CSS for avatars and overall styling.
- `terraform/`: Contains Terraform scripts to create AWS infrastructure.
- `build/`: Auto-generated production-ready React files.

5. Code Explanation with Screenshots

This section explains the key code files in the project, including Terraform scripts for AWS infrastructure and the main React component.

5.1 Terraform Code

main.tf



```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

resource "aws_s3_bucket" "website" {
  bucket        = var.bucket_name
  force_destroy = true
}

resource "aws_s3_bucket_website_configuration" "website" {
  bucket = aws_s3_bucket.website.id

  index_document {
    suffix = "index.html"
  }

  error_document {
    key = "index.html"
  }
}

resource "aws_s3_bucket_public_access_block" "website" {
  bucket = aws_s3_bucket.website.id

  block_public_acls       = false
  block_public_policy     = false
  ignore_public_acls      = false
  restrict_public_buckets = false
}
```

```
resource "aws_s3_bucket_policy" "website" {

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid       = "PublicReadGetObject"
        Effect    = "Allow"
        Principal = "*"
        Action    = "s3:GetObject"
        Resource  = "${aws_s3_bucket.website.arn
      },
    ]
  })

  depends_on = [aws_s3_bucket_public_access_bloc

}

resource "aws_cloudfront_distribution" "website"
  origin {
    domain_name = aws_s3_bucket_website_configur
    origin_id   = "S3-${var.bucket_name}"

    custom_origin_config {
      http_port              = 80
      https_port             = 443
      origin_protocol_policy = "http-only"
      origin_ssl_protocols   = ["TLSv1.2"]
    }
  }

  enabled             = true
  default_root_object = "index.html"

  default_cache_behavior {
    allowed_methods        = ["DELETE", "GET",
    cached_methods         = ["GET", "HEAD"]
    target_origin_id       = "S3-${var.bucket_na
    compress               = true
```

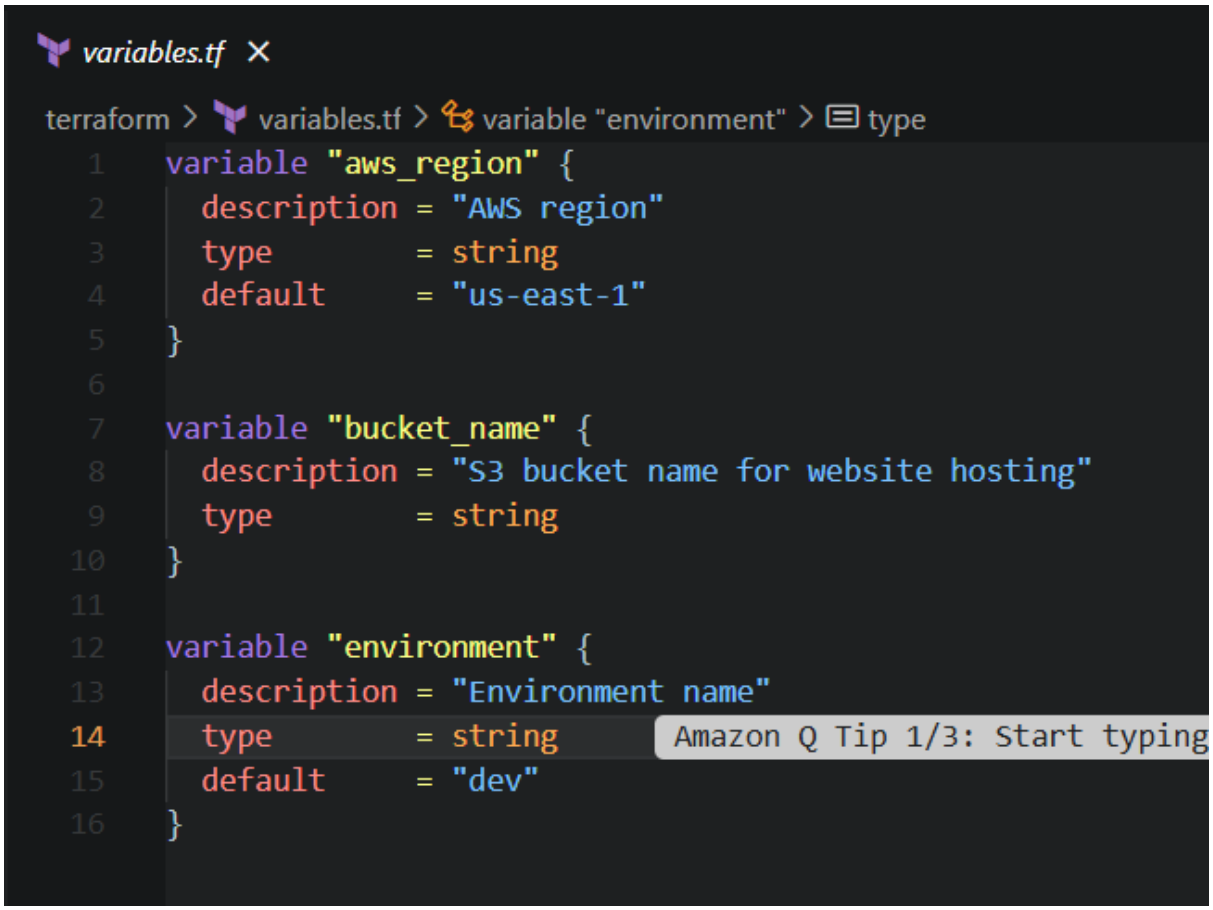
```
resource "aws_cloudfront_distribution" "website"
  default_cache_behavior {
    compress               = true
    viewer_protocol_policy = "redirect-to-https"

    forwarded_values {
      query_string = false
      cookies {
        forward = "none"
      }
    }
  }

  custom_error_response {
    error_code         = 404
    response_code      = 200
    response_page_path = "/index.html"
  }

  restrictions {
    geo_restriction {
      restriction_type = "none"
    }
  }

  viewer_certificate {
    cloudfront_default_certificate = true
  }
}
```

Explanation:

- **Provider block:** Specifies AWS as the provider and sets the region.
- **S3 Bucket:** Hosts React build files and enables static website hosting.
- **CloudFront Distribution:** Serves the S3 content globally with HTTPS.
- **Default Cache Behavior:** Configures HTTP methods, caching, and security.

---

variables.tf



Explanation:

- Stores reusable configuration variables for Terraform.
- `aws_region` sets the deployment region.
- `s3_bucket_name` sets the name of the S3 bucket.

---

outputs.tf

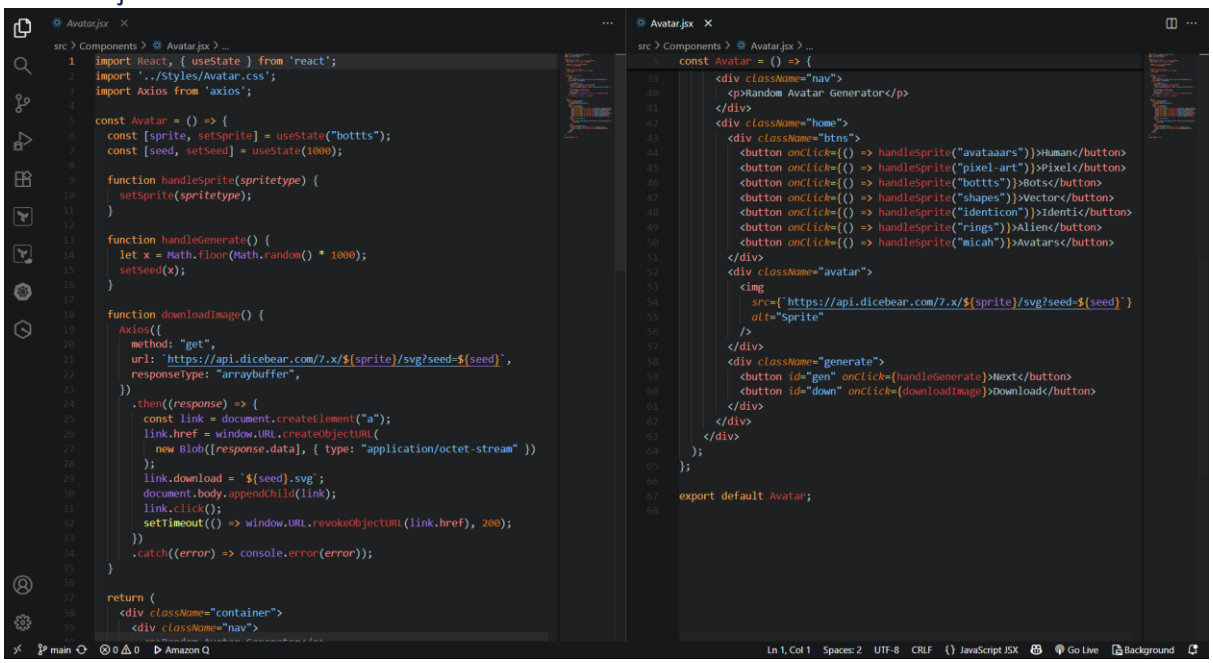
```hcl
output "website_url" {
  description = "Website URL"
  value       = "https://${aws_cloudfront_distribution.website.domain_name}"
}

output "s3_bucket_name" {
  description = "S3 bucket name"
  value       = aws_s3_bucket.website.bucket
}

output "cloudfront_distribution_id" {
  description = "CloudFront distribution ID"
  value       = aws_cloudfront_distribution.website.id
}
```

Explanation:

- Outputs the CloudFront URL after Terraform deployment.
- Allows easy access to the live website.

5.2 React Component

Avatar.jsx



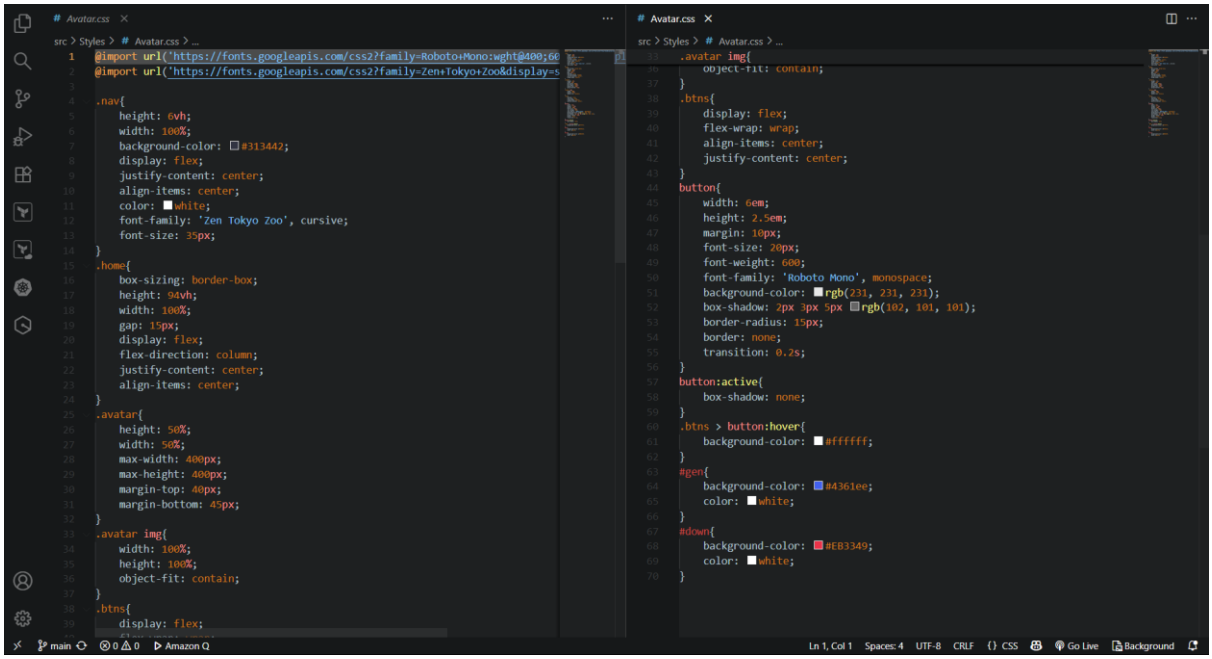
```jsx
import React, { useState } from 'react';
import '../Styles/Avatar.css';
import Axios from 'axios';

const Avatar = () => {
  const [sprite, setSprite] = useState("bottts");
  const [seed, setSeed] = useState(1000);

  function handleSprite(spritetype) {
    setSprite(spritetype);
  }

  function handleGenerate() {
    let x = Math.floor(Math.random() * 1000);
    setSeed(x);
  }

  function downloadImage() {
    Axios({
      method: "get",
      url: `https://api.dicebear.com/7.x/${sprite}/svg?seed=${seed}`,
      responseType: "arraybuffer",
    })
      .then((response) => {
        const link = document.createElement("a");
        link.href = window.URL.createObjectURL(
          new Blob([response.data], { type: "application/octet-stream" })
        );
        link.download = `${seed}.svg`;
        document.body.appendChild(link);
        link.click();
        setTimeout(() => window.URL.revokeObjectURL(link.href), 200);
      })
      .catch((error) => console.error(error));
  }

  return (
    <div className="container">
      <div className="nav">
        <p>Random Avatar Generator</p>
      </div>
      <div className="home">
        <div className="btns">
          <button onClick={() => handleSprite("avataaars")}>Human</button>
          <button onClick={() => handleSprite("pixel-art")}>Pixel</button>
          <button onClick={() => handleSprite("bottts")}>Bots</button>
          <button onClick={() => handleSprite("shapes")}>Vector</button>
          <button onClick={() => handleSprite("identicon")}>Identic</button>
          <button onClick={() => handleSprite("rings")}>Alien</button>
          <button onClick={() => handleSprite("micah")}>Avatars</button>
        </div>
        <div className="avatar">
          <img
            src={`https://api.dicebear.com/7.x/${sprite}/svg?seed=${seed}`}
            alt="sprite"
          />
        </div>
        <div className="generate">
          <button id="gen" onClick={handleGenerate}>Next</button>
          <button id="down" onClick={downloadImage}>Download</button>
        </div>
      </div>
    </div>
  );
};

export default Avatar;
```

Explanation:

- Receives `name` and `image` as props.

- Displays the avatar image and name inside a styled card.
- Exported to be used in other parts of the app.

Avatar.css



```
@import url('https://fonts.googleapis.com/css2?family=Roboto+Mono:wght@400;60
@import url('https://fonts.googleapis.com/css2?family=Zen+Tokyo+Zoo&display=s

.nav{
    height: 6vh;
    width: 100%;
    background-color: #313442;
    display: flex;
    justify-content: center;
    align-items: center;
    color: white;
    font-family: 'Zen Tokyo Zoo', cursive;
    font-size: 35px;
}
.home{
    box-sizing: border-box;
    height: 94vh;
    width: 100%;
    gap: 15px;
    display: flex;
    flex-direction: column;
    justify-content: center;
    align-items: center;
}
.avatar{
    height: 50%;
    width: 50%;
    max-width: 400px;
    max-height: 400px;
    margin-top: 40px;
    margin-bottom: 45px;
}
.avatar img{
    width: 100%;
    height: 100%;
    object-fit: contain;
}
.btns{
    display: flex;
```

```
.avatar img{
    object-fit: contain;
}
.btns{
    display: flex;
    flex-wrap: wrap;
    align-items: center;
    justify-content: center;
}
button{
    width: 6em;
    height: 2.5em;
    margin: 10px;
    font-size: 20px;
    font-weight: 600;
    font-family: 'Roboto Mono', monospace;
    background-color: rgb(231, 231, 231);
    box-shadow: 2px 3px 5px rgb(102, 101, 101);
    border-radius: 15px;
    border: none;
    transition: 0.2s;
}
button:active{
    box-shadow: none;
}
.btns > button:hover{
    background-color: #ffffff;
}
#gen{
    background-color: #4361ee;
    color: white;
}
#down{
    background-color: #EB3349;
    color: white;
}
```

Explanation:

- **.avatar-card:** Styles the container with margin, padding, and shadow.
- **.avatar-image:** Makes the image circular and fixed size.
- **.avatar-name:** Styles the text for the avatar's name.

6. Local Development

To run the project locally:

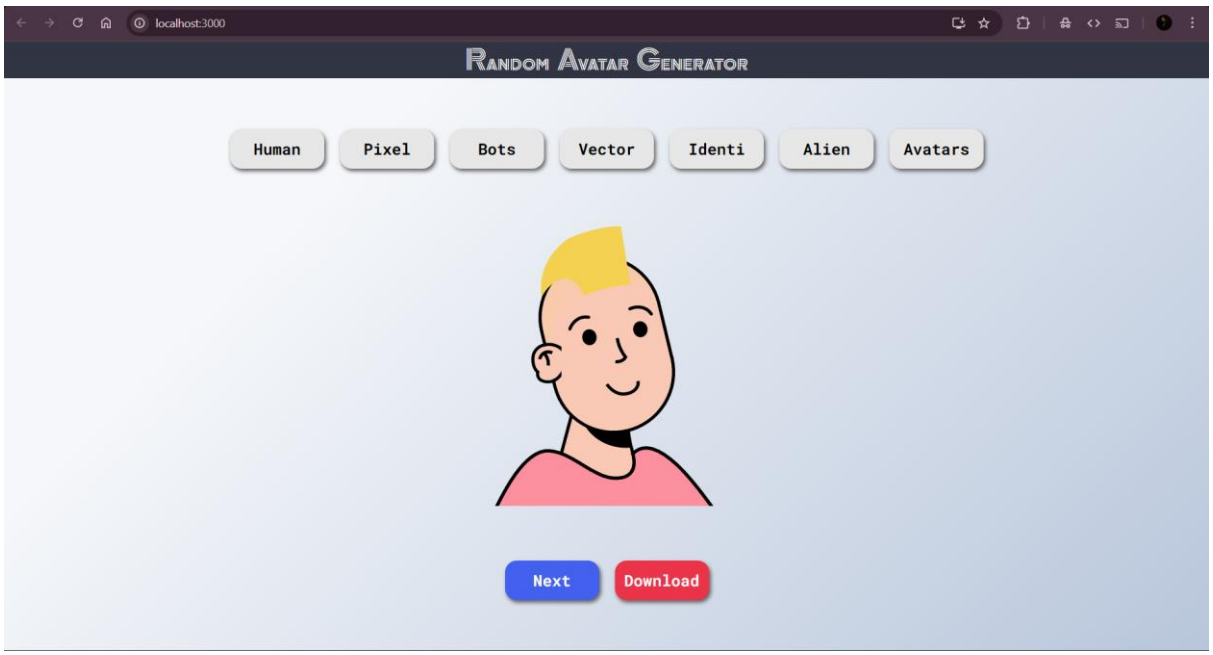
1. Install dependencies:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp (main)
$ npm install

up to date, audited 1344 packages in 5s

271 packages are looking for funding
  run `npm fund` for details

9 vulnerabilities (3 moderate, 6 high)

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
```

2. Start development server:

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp (main)
$ npm start

> avatarapp@0.1.0 start
> react-scripts start

(node:12652) [DEP_WEBPACK_DEV_SERVER_ON_AFTER_SETUP_MIDDLEWARE] DeprecationWarning: '
onAfterSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' optio
n.
(Use `node --trace-deprecation ...` to show where the warning was created)
(node:12652) [DEP_WEBPACK_DEV_SERVER_ON_BEFORE_SETUP_MIDDLEWARE] DeprecationWarning:
'onBeforeSetupMiddleware' option is deprecated. Please use the 'setupMiddlewares' opt
ion.
Starting the development server...
Compiled successfully!

You can now view avatarapp in the browser.

  Local:            http://localhost:3000
  On Your Network:  http://192.168.56.1:3000

Note that the development build is not optimized.
To create a production build, use npm run build.

webpack compiled successfully
```

3. Open `http://localhost:3000` in your browser.



7. AWS Deployment Steps

Step 1: Setup Infrastructure using Terraform

- **`cd terraform`**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp (main)
$ cd terraform
```

- **aws configure # Ensure AWS CLI is configured**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ aws configure
AWS Access Key ID [****************ACHK]:
AWS Secret Access Key [****************2TDX]:
Default region name [us-east-1]:
Default output format [json]:
```

- **terraform init # Initialize Terraform**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ terraform init
Initializing the backend...
Initializing provider plugins...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)
Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
```

- **terraform fmt # To check the format**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ terraform fmt
```

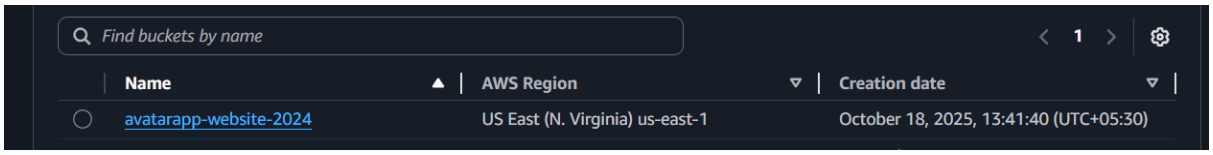
- **terraform plan # To plan the terraform**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ terraform plan
var.bucket_name
  S3 bucket name for website hosting

  Enter a value: avatar1224


Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_cloudfront_distribution.website will be created
  + resource "aws_cloudfront_distribution" "website" {
      + arn                            = (known after apply)
      + caller_reference               = (known after apply)
      + continuous_deployment_policy_id = (known after apply)
      + default_root_object            = "index.html"
```

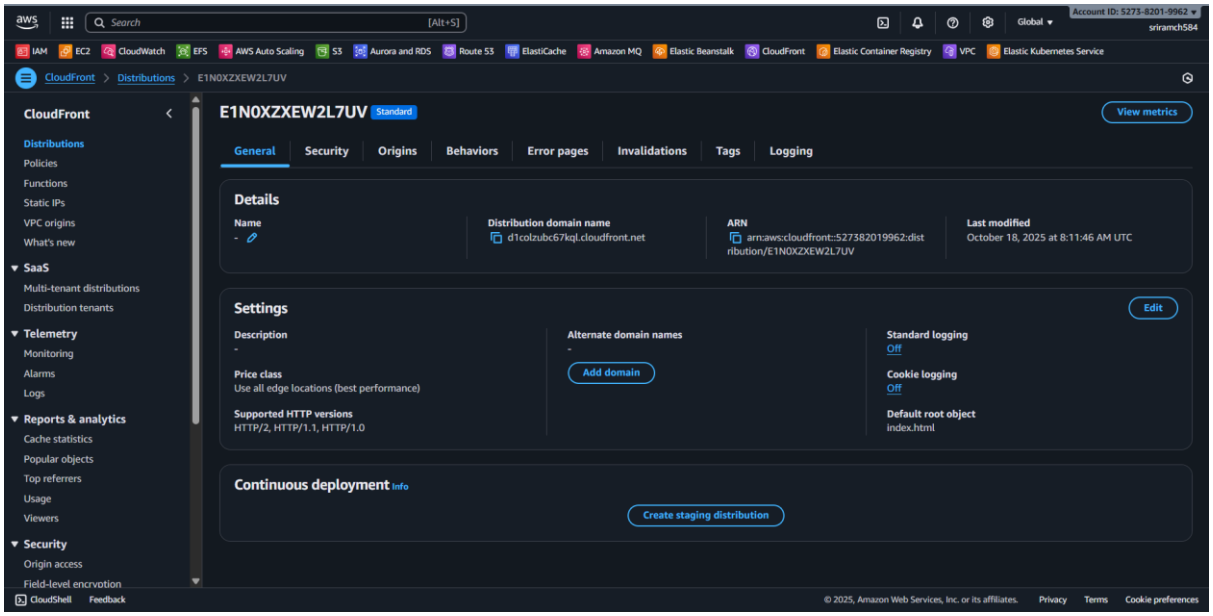
- `terraform apply` `# Deploy S3 + CloudFront`

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ terraform apply

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:
  + create

Terraform will perform the following actions:

  # aws_cloudfront_distribution.website will be created
  + resource "aws_cloudfront_distribution" "website" {
      + arn                            = (known after apply)
      + caller_reference               = (known after apply)
      + continuous_deployment_policy_id = (known after apply)
      + default_root_object            = "index.html"
      + domain_name                    = (known after apply)
      + enabled                        = true
      + etag                           = (known after apply)
      + hosted_zone_id                 = (known after apply)
      + http_version                   = "http2"
      + id                             = (known after apply)
      + in_progress_validation_batches  = (known after apply)
      + is_ipv6_enabled                = false
      + last_modified_time             = (known after apply)
      + price_class                    = "PriceClass_All"
      + retain_on_delete               = false
      + staging                        = false
      + status                         = (known after apply)
      + tags_all                       = (known after apply)
      + trusted_key_groups             = (known after apply)
      + trusted_signers                = (known after apply)
      + wait_for_deployment            = true
```

- Creates S3 bucket.



- CloudFront distribution



- Gives outputs `website_url`.

```
Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

cloudfront_distribution_id = "E1N0XZXEW2L7UV"
s3_bucket_name = "avatarapp-website-2024"
website_url = "https://d1colzubc67kql.cloudfront.net"
```

Step 2: Deploy Website

- **`cd ..`**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ cd ..
```

- **npm run build # Create production build**

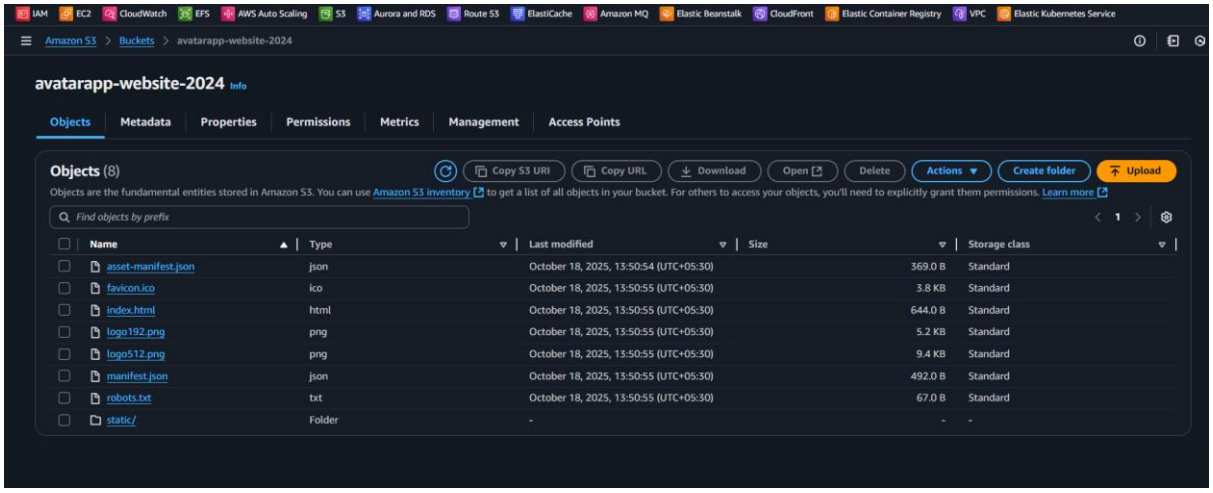
```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp (main)
$ npm run build

> avatarapp@0.1.0 build
> react-scripts build

Creating an optimized production build...
Compiled successfully.

File sizes after gzip:

  75.13 kB  build\static\js\main.6002cd38.js
  780 B     build\static\css\main.a234a338.css

The project was built assuming it is hosted at /.
You can control this with the homepage field in your package.json.

The build folder is ready to be deployed.
You may serve it with a static server:

  npm install -g serve
  serve -s build

Find out more about deployment here:

  https://cra.link/deployment
```

- **cd terraform**

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp (main)
$ cd terraform
```

- Syncs React build folder with S3 bucket.

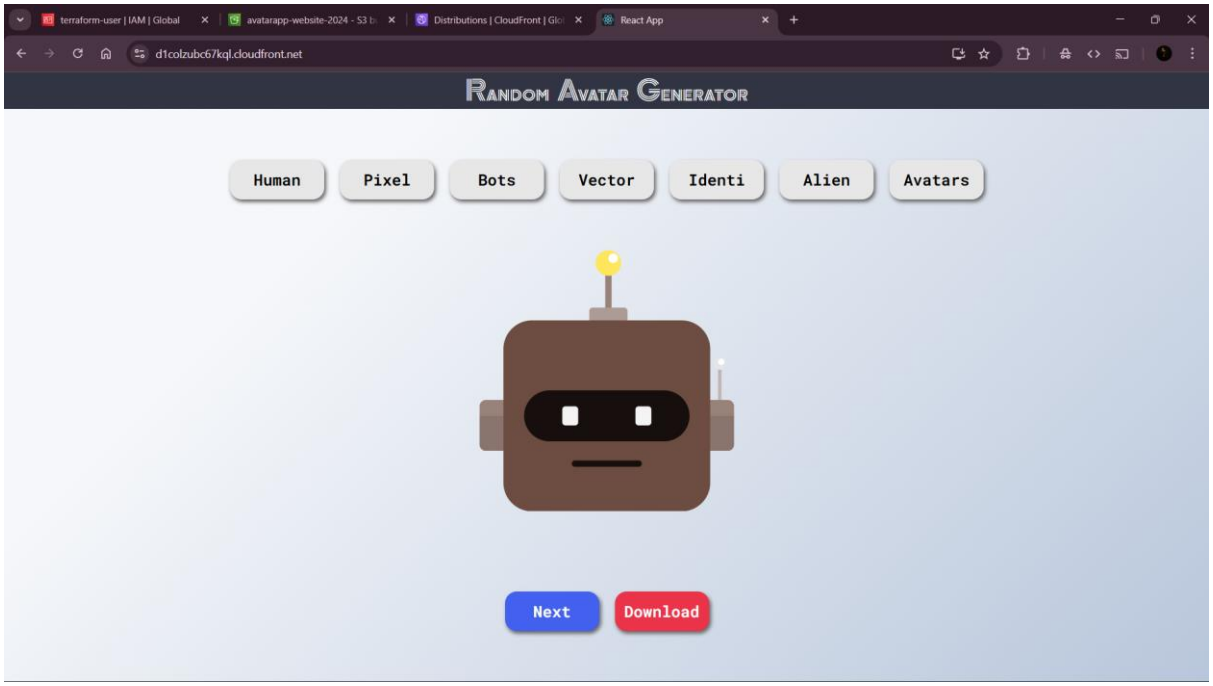
```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ aws s3 sync ../build/ s3://avatarapp-website-2024 --delete
Completed 369 Bytes/~24.3 KiB (710 Bytes/s) with ~9 file(s) remaining (calculating...
upload: ..\build\asset-manifest.json to s3://avatarapp-website-2024/asset-manifest.js
on
Completed 369 Bytes/~24.3 KiB (710 Bytes/s) with ~8 file(s) remaining (calculating...
Completed 3.3 KiB/1.3 MiB (1.5 KiB/s) with 11 file(s) remaining
upload: ..\build\static\css\main.a234a338.css.map to s3://avatarapp-website-2024/stat
ic/css/main.a234a338.css.map
upload: ..\build\favicon.ico to s3://avatarapp-website-2024/favicon.ico
upload: ..\build\manifest.json to s3://avatarapp-website-2024/manifest.json
upload: ..\build\static\js\main.6002cd38.js to s3://avatarapp-website-2024/static/js/
main.6002cd38.js
upload: ..\build\static\css\main.a234a338.css to s3://avatarapp-website-2024/static/c
ss/main.a234a338.css
upload: ..\build\robots.txt to s3://avatarapp-website-2024/robots.txt
upload: ..\build\logo192.png to s3://avatarapp-website-2024/logo192.png
upload: ..\build\logo512.png to s3://avatarapp-website-2024/logo512.png
upload: ..\build\index.html to s3://avatarapp-website-2024/index.html
upload: ..\build\static\js\main.6002cd38.js.LICENSE.txt to s3://avatarapp-website-202
4/static/js/main.6002cd38.js.LICENSE.txt
upload: ..\build\static\js\main.6002cd38.js.map to s3://avatarapp-website-2024/static
/js/main.6002cd38.js.map
```



Step 3: Access Live URL

- Provides the CloudFront URL for the live website.

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ terraform output website_url
"https://d1colzubc67kql.cloudfront.net"
```



---

8. Destroy Infrastructure

To remove AWS resources:

- `cd terraform`

- `aws s3 rm s3://avatarapp-website-2024 –recursive`

```
CHANDIKA SRIRAM@CHANDIKA-SRIRAM MINGW64 ~/Desktop/Terraform/avatarapp/terraform (main
)
$ aws s3 rm s3://avatarapp-website-2024 --recursive
delete: s3://avatarapp-website-2024/static/css/main.a234a338.css
delete: s3://avatarapp-website-2024/robots.txt
delete: s3://avatarapp-website-2024/manifest.json
delete: s3://avatarapp-website-2024/logo512.png
delete: s3://avatarapp-website-2024/favicon.ico
delete: s3://avatarapp-website-2024/static/css/main.a234a338.css.map
delete: s3://avatarapp-website-2024/index.html
delete: s3://avatarapp-website-2024/asset-manifest.json
delete: s3://avatarapp-website-2024/static/js/main.6002cd38.js
delete: s3://avatarapp-website-2024/logo192.png
delete: s3://avatarapp-website-2024/static/js/main.6002cd38.js.LICENSE.txt
delete: s3://avatarapp-website-2024/static/js/main.6002cd38.js.map
```

- `terraform destroy`

- Removes S3 bucket and CloudFront distribution.

```
aws_s3_bucket_policy.website: Destroying... [id=avatarapp-website-2024]
aws_cloudfront_distribution.website: Destroying... [id=E1NOXZXEW2L7UV]
aws_s3_bucket_policy.website: Destruction complete after 1s
aws_s3_bucket_public_access_block.website: Destroying... [id=avatarapp-website-2024]
aws_s3_bucket_public_access_block.website: Destruction complete after 0s
```

- Prevents recurring AWS charges.

Screenshot placeholder: Terraform destroy console screenshot.

---

9. Cost Overview

- **S3 Bucket:** ~$0.023 per GB/month
- **CloudFront:** Free tier (1TB transfer)
- **Total:** Usually under $1/month for low-traffic deployment

---

10. Conclusion

The **Avatar App** project demonstrates:

- Building interactive React applications.
- Deploying production-ready websites on AWS.
- Automating infrastructure with Terraform.
- Maintaining scalable, secure, and reproducible deployments.

This documentation provides a **step-by-step guide** for developers to replicate the environment, test locally, and deploy live on AWS confidently.